RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

GRENOBLE INP UGA Ensimag

École nationale supérieure d'informatique et de mathématiques appliquées

Cours 4

Test et spécifications fonctionnelles 2

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

GRENOBLE INP UGA Ensimag

École nationale supérieure d'informatique et de mathématiques appliquées

Model-Based Testing and Testing based on Finite State Machines

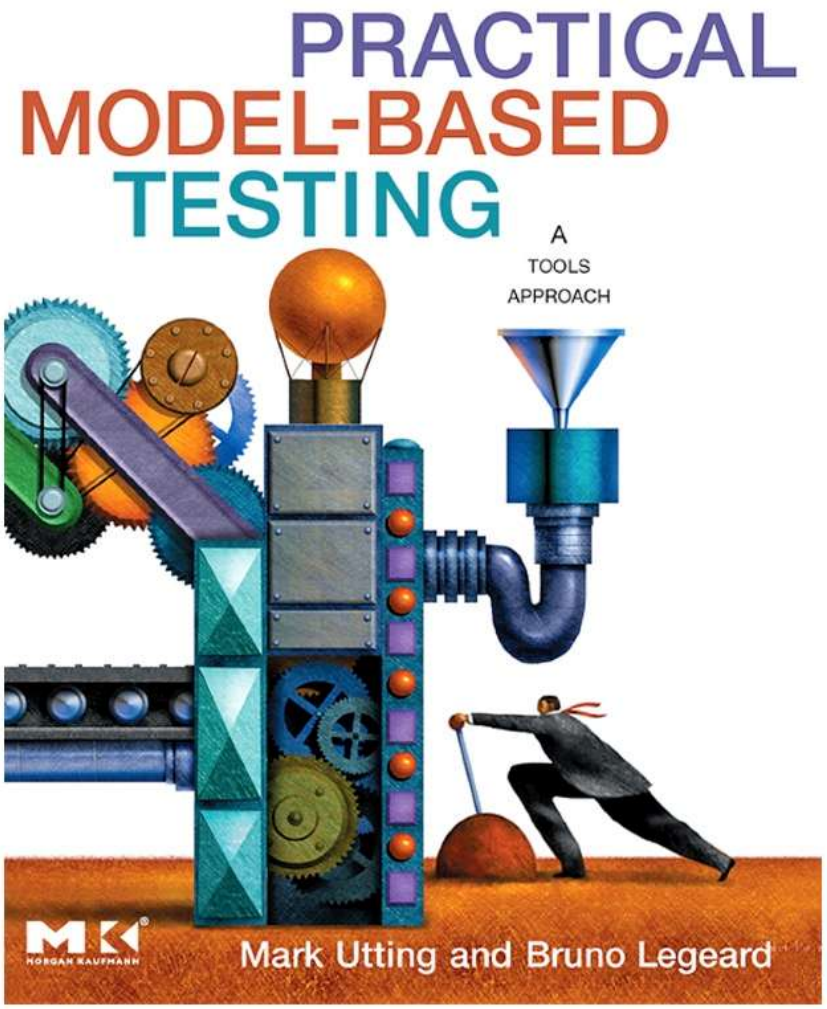
Roland GROZ

LIG, Université de Grenoble, France

Outline

- MBT and software engineering
- Conformance testing & BB identification
  - Conformance relations, Fault models, TT
- Basic notions on Finite State Machines
  - DS, UIO, W-set
- FSM testing
- Other MBT methods and tools

Model-Based Testing

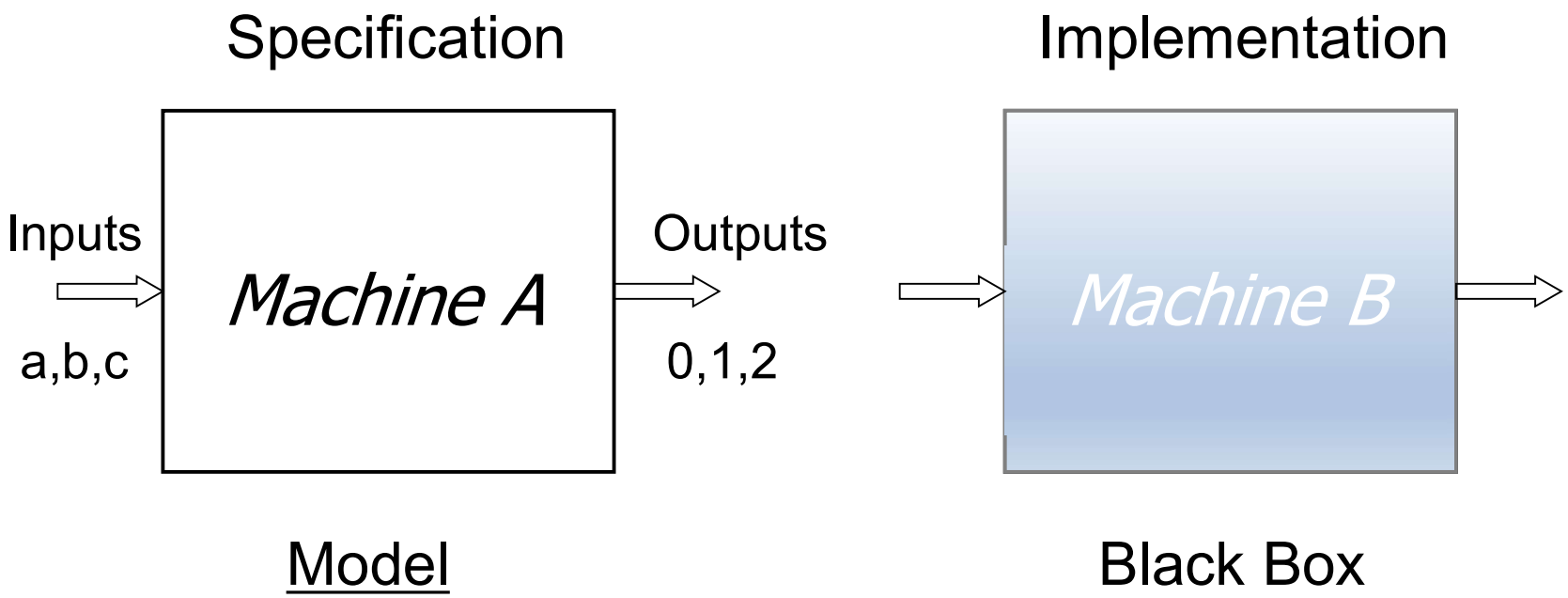


- *Model-based testing is the automation of the design of black-box tests.*
- Deriving tests from behavioural models
  - UML state charts
  - B-machines
  - …
- Test cases with selected inputs and expected outputs

Behavioural models

- To test sequences of events
  - not just isolated calls
  - models define allowed sequences, and expected outputs
- Black-box models:
  - events: inputs and outputs
  - Finite State Machines (Mealy I/O automata)
    - or I O transition systems (IOTS, IOLTS, IOSM, UML/Statecharts etc...)

Conformance testing



Does B conform to A ?

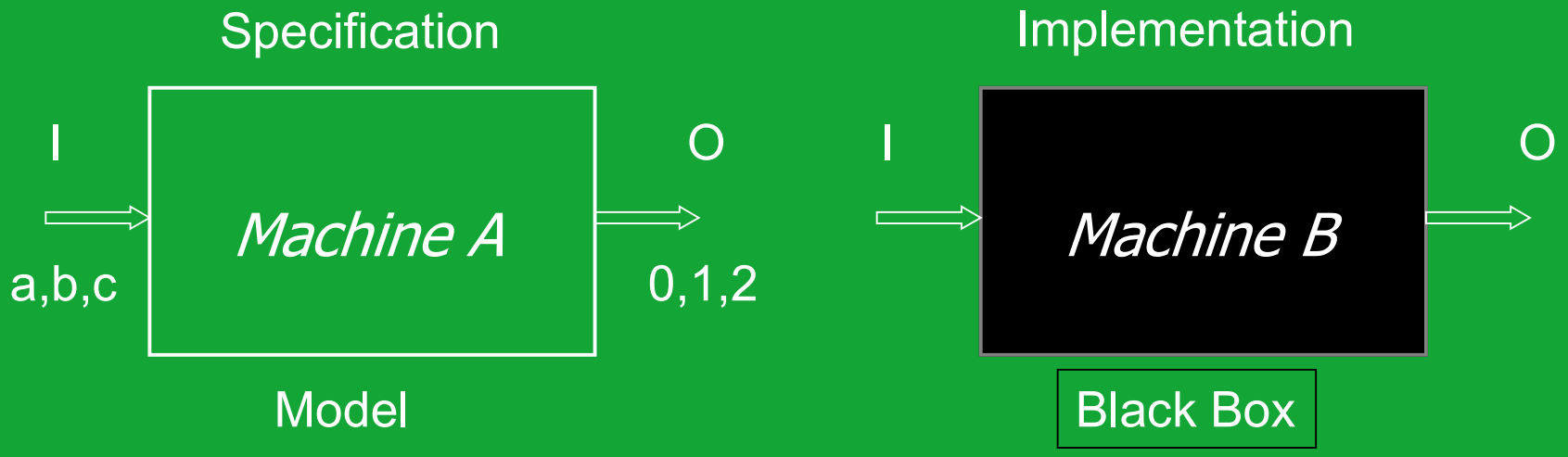
i.e. does it BEHAVE like A ?

Behaviours as traces

Example of I/O trace: a 1 b 0 a 2 c 0 a 1 2 c b 0

- Tr(A) = set of all traces that machine A can exhibit (from its initial state)
  - Similar to L(A) = language: set of words accepted by a DFA
  - Except here no notion of accepting state, but prefix and current input restrict next potential output(s)

Which conformance relation ?



Example of I/O trace: a 1 b 0 a 2 c 0 a 1 2 c b 0

- Trace equivalence: Tr(A) = Tr(B)
- Trace inclusion: Tr(B) $\subseteq$ Tr(A)
- …

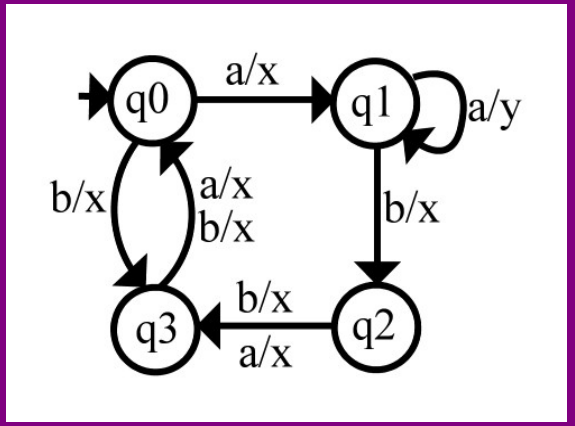
Usual conformance relations

- Trace inclusion: $Tr(B) \subseteq Tr(A)$
  - all behaviours of B must be in A
  - A can allow more behaviours (B has done implementation choices)
- ioco: $\mathbf{B\ ioco\ A =_{def} \forall\ \sigma \in Tr(A), \forall\ i \in I :\ out\ (B\ after\ \sigma.i) \subseteq out\ (A\ after\ \sigma.i)}$
  - As long as B as behaved as A, B's outputs are among those allowed by A
  - But B can accept inputs unspecified by A

Model considered & definitions

- **Mealy Machine**: $M = (Q, I, O, \delta, \lambda, q_0)$
  - Q : *set of states*
  - I : *set of input symbols*
  - O : *set of output symbols*
  - δ : *transition function*
  - λ : *output function*
  - q_0 : *initial state*

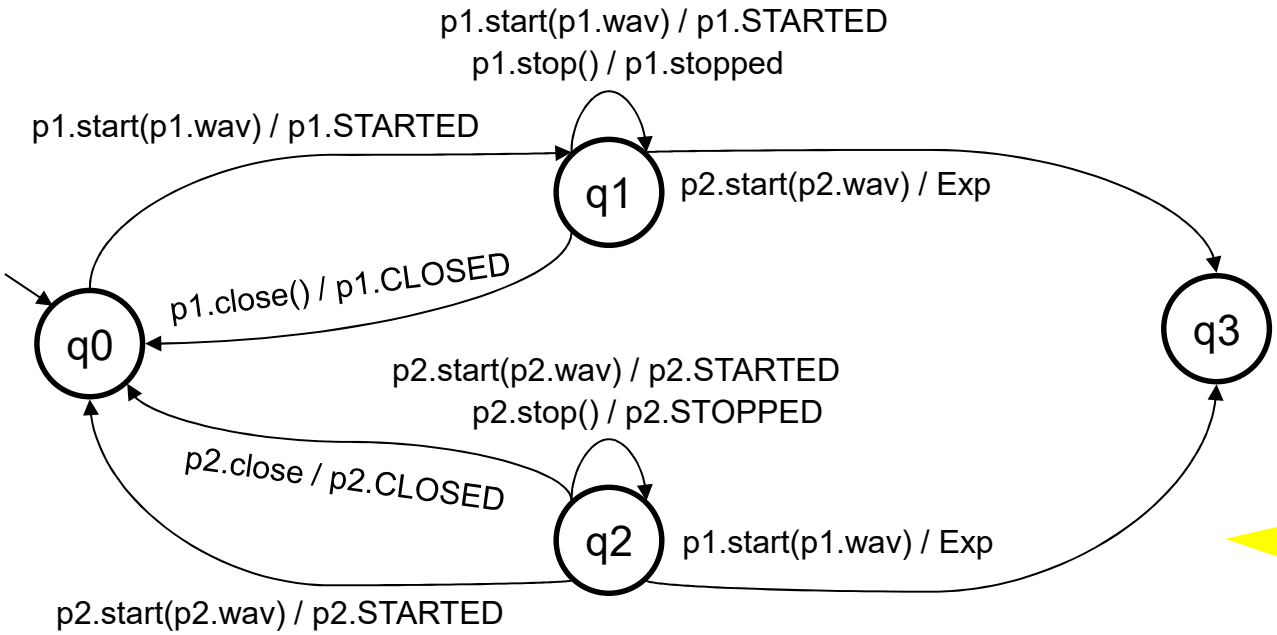
- Deterministic: δ, λ *are functions*
- Input Enabled (aka complete)
  - $\mathrm{dom}(\delta) = \mathrm{dom}(\lambda) = \mathrm{Q} \times \mathrm{I}$
  - Usually implementations input-enabled (cannot refuse) but specs can be incomplete
- Minimal: no two states (trace-)equivalent
- Reset: extra input "r" to bring M to q_0



- $\mathrm{Q} = \{q_0, q_1, q_2, q_3\}$
- $\mathrm{I} = \{a,b\}$
- $\mathrm{O} = \{x,y\}$

Example

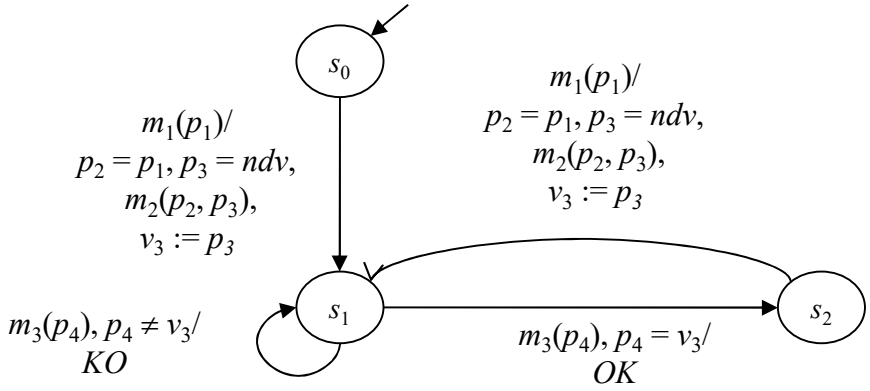
FSM as abstractions of complex s/w



Music
P1.wav
Music
P2.wav

{ Play,
Pause,
Stop }

Nokia 6131
(PFSM)



Needham-Schroeder crypto protocol
with nonces (NonDeterministicValues), guards etc

Conformance for Mealy models

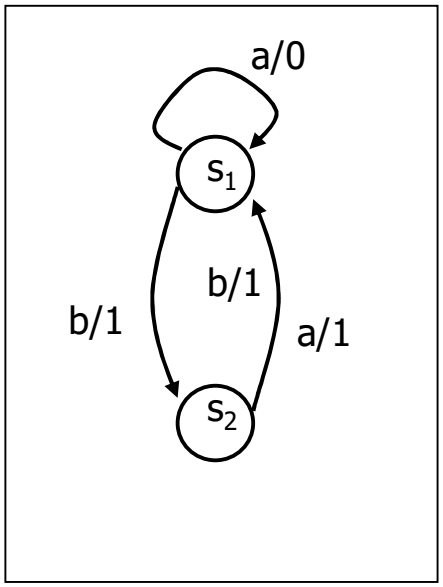
- If spec. A and implem. B are both modelled as Mealy machines, conformance is (usually):
  - trace equivalence if A is complete (and deterministic):

    $B \text{ conf } A \Leftrightarrow B \approx A \Leftrightarrow Tr(B) = Tr(A)$

  - trace inclusion for ND (Non-Deterministic) specs
- If A is minimal, then equivalence means B is isomorphic to A (i.e. equal up to state renaming)

Example

Specification A



(Input) Test sequence

a a b a b b

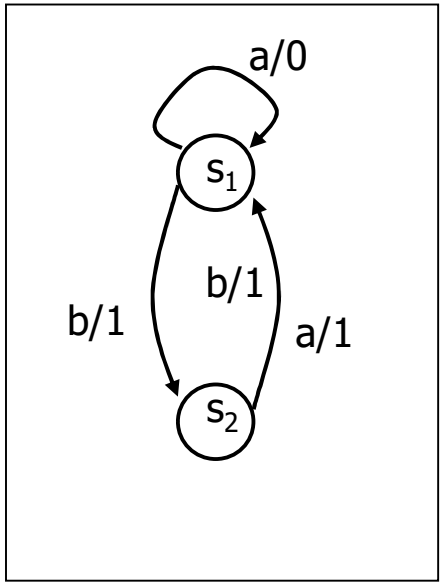
Expected output sequence

? 0 1 1 1 1

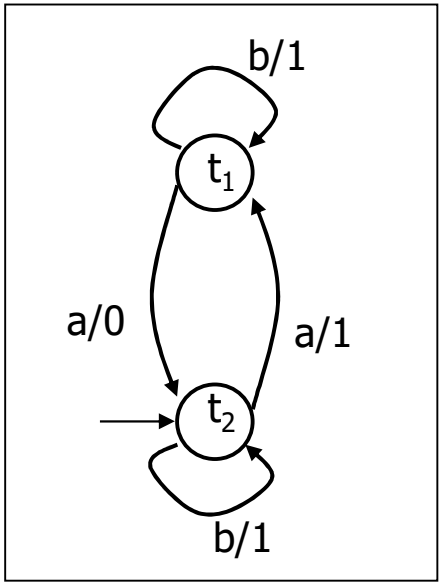
Is this test OK/enough ?

Incomplete testing

Specification A



Machine B



(Input) Test sequence

a a b a b b

Expected output seq.

? 0 1 1 1 1

Soundness & exhaustiveness

- A test suite T is *sound* iff every implementation B that conforms to spec. A passes (succeeds) T
  - *T will never fail on correct systems*
- A test suite T is *exhaustive* iff every implementation B that passes (succeeds) T conforms to A
  - *each incorrect system will be detected by T*
- Complete = sound + exhaustive

Achieving completeness

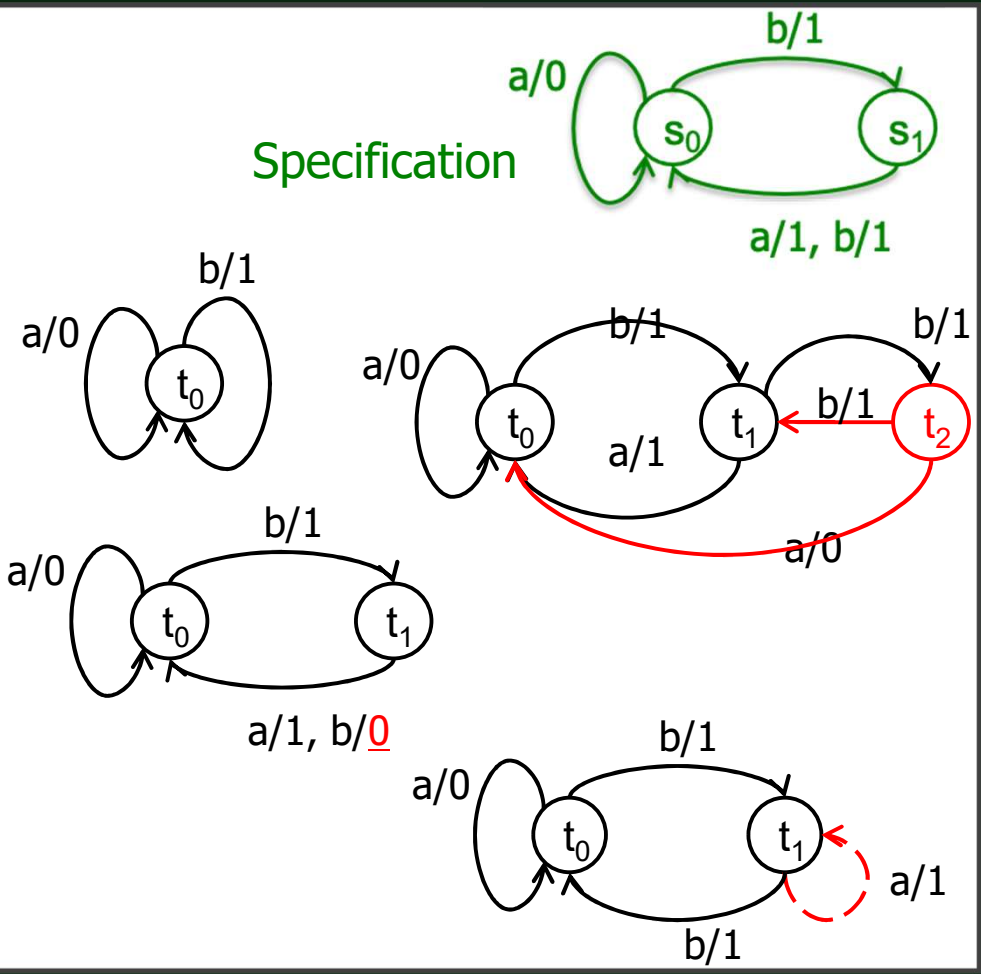
- Soundness: easy to achieve
  - By automatic test case generation
  - Note: Manual test suites are error prone
- Exhaustiveness: usually impossible
  - Can be achieved on restricted fault domains
    - not necessarily very restrictive, but well defined
  - Again: competent programmer hypothesis

Fault model for FSM

Causes for non-isomorphism

(hence non – conformance)

- Different number of states
- Transition differs:
  - wrong output (output fault)
  - wrong target (transfer fault)



Towards exhaustiveness

- Fault models
  - Programmers make kinds of mistakes
  - Resulting implementation can be *modelled* by a faulty model of behaviour
- FSM models are simple
  - Faulty models can be enumerated
  - Restricted types of (single) faults

1st test generation method for FSM: Transition Tour

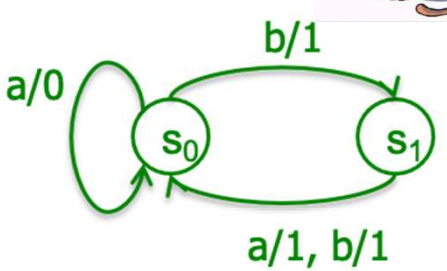
- Assessing correct implementation of spec

by

- Traversing (covering) all nodes and edges

- Uses algorithms on graph: optimal graph traversal

Transition tour

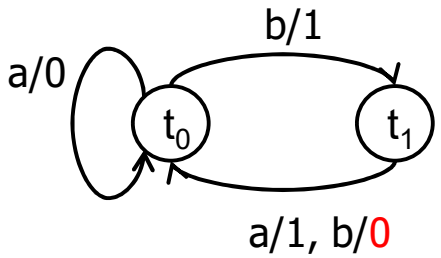


- FSM = graph
- Test suite: a tour, i.e. a path that traverses at least once every edge and node.
- Algo on graph: « Rural Chinese Postman »

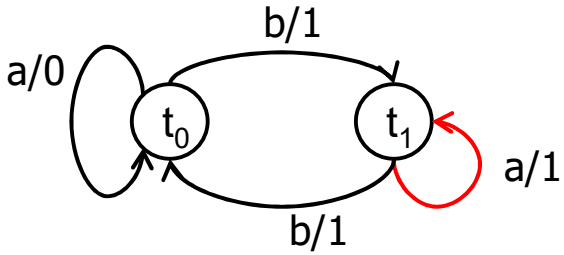
- Based on spec
- Analogous to branch coverage (but here: dynamic graph)

Transition tour

- Will detect all output faults



- May miss transfer faults
  - fault masking:
    t1 and s1 cannot be distinguished by just a, b or abb or bba



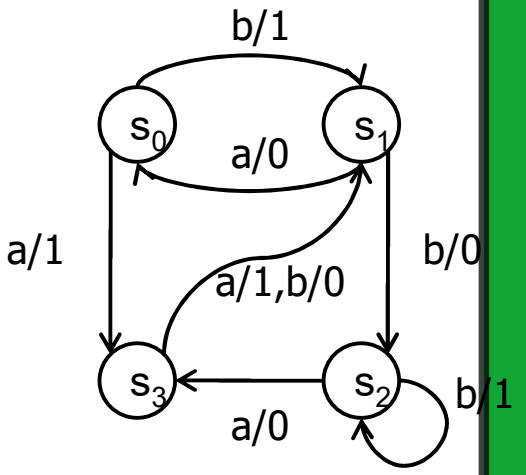
24

Improving on Transition Tour

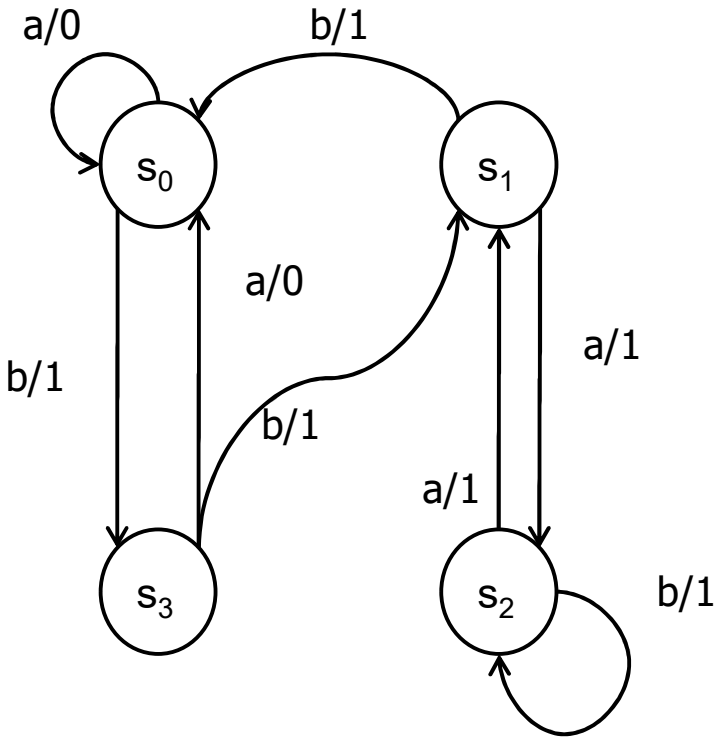
- BB testing: internal states cannot be observed directly
- Checking a transition:
  - output can be observed
  - how can we *check* the *target* state ?
- Spec minimal => target states inequivalent => can be distinguished by a future trace
  - How should we choose such traces ?

DS: Distinguishing Sequence

- FSM M has a DS $\in I^*$: DS= $i_1\ i_2 \ldots i_k$ iff the corresponding output sequence $Tr(s_j,DS)\downarrow_O$ is different for each state $s_j \in Q$
- Ex: bb is a DS
  - s_0: b/1 b/0
  - s_1: b/0 b/1
  - s_2: b/1 b/1
  - s_3: b/0 b/0



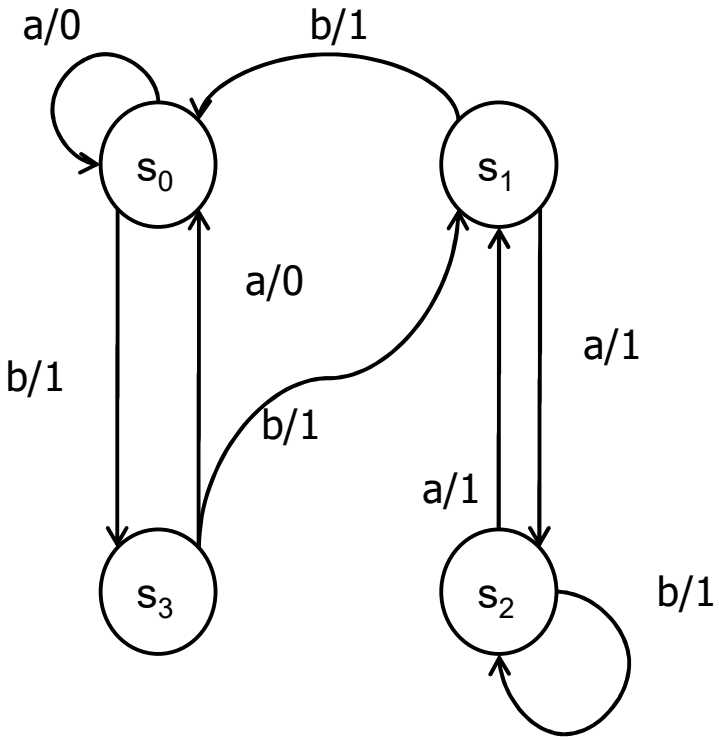
Exercice: can you find a DS ?



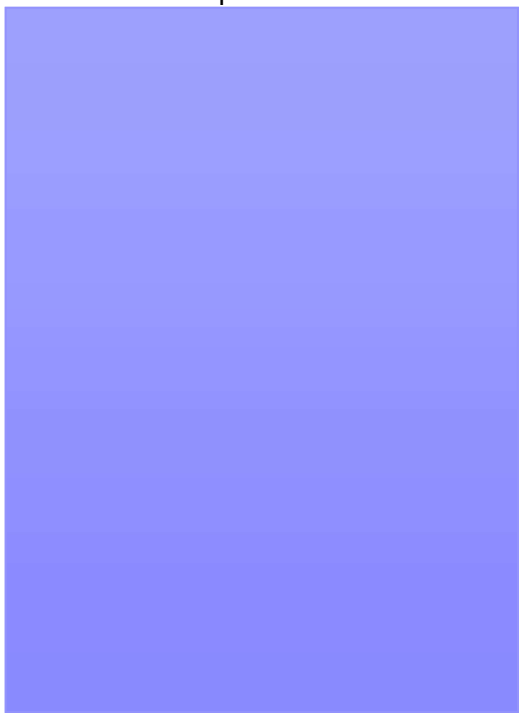
UIO: Unique Input Output seq.

- FSM M has a UIO $\in I^*$ for state $s_j \in Q$
  $UIO_j = i_1 \ldots i_k$
  such that $Tr(s_j, UIO_j)\downarrow_O = o_1 \ldots o_k$
  iff the corresponding output sequence for each state $s_i \neq s_j$
  $Tr(s_i, UIO_j)\downarrow_O = o'_1 \ldots o'_k$
  is different from $o_1 \ldots o_k$
- UIO_j : signature for state s_j .

Exercice: can you find UIOs ?

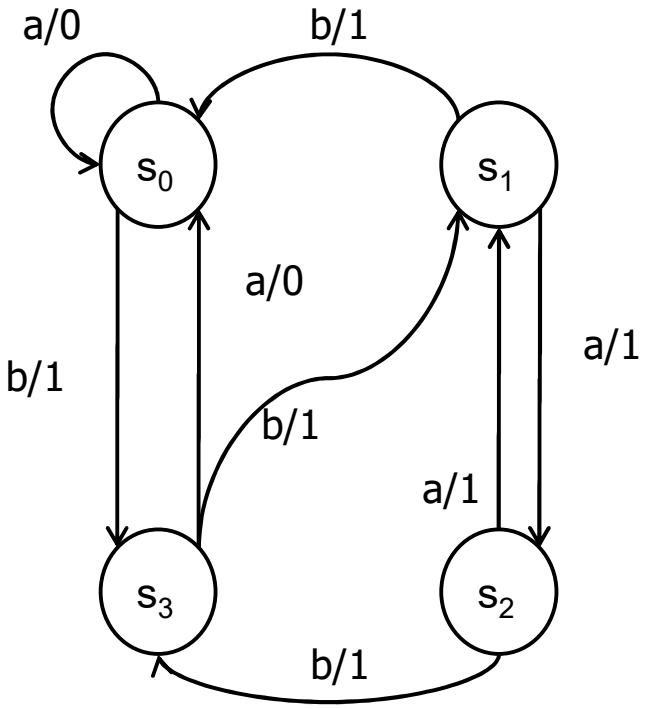


UIO for s_1:



UIO for s_0: ?

Exercice: can you find UIOs ?

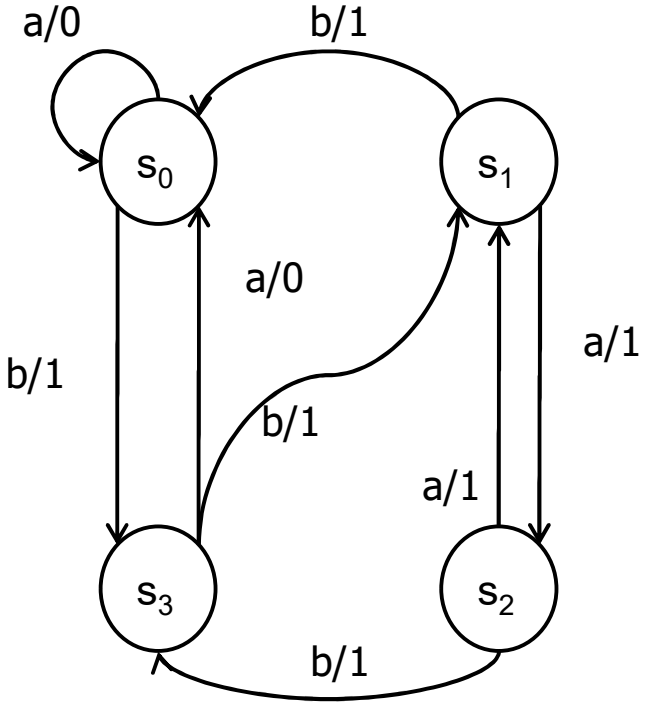


UIO for s_1:

Characterizing set: W-set

- If M is minimal: for any pair of states (s_i,s_j), $s_i \neq s_j \Rightarrow Tr(s_i) \neq Tr(s_j)$
- So there exists one sequence d_{ij} distinguishing them
- $\{d_{ij}\}_{i \neq j}$ is a set of sequences that characterize states
- A W-set is a characterizing set
  - $\{d_{ij}\}_{i \neq j}$ is a long one (up to n(n-1)/2 sequences)
  - We can look for shorter (optimal) W-sets

Example: W-set



W = { a , bba}

s_0: a/0 , b/1 b/1 a/1

s_1: a/1 , b/1 b/1 a/0

s_2: a/1 , b/1 b/1 a/1

s_3: a/0 , b/1 b/1 a/0

Summing up

- In a minimal machine, states can be distinguished
- DS: a single sequence can distinguish all,
  but a DS may not exist
- When no DS exists, different UIOs can recognize states, but may not exist
- In all cases, it is possible to distinguish states with a W-set
  NB: if there is a DS, then W={DS}

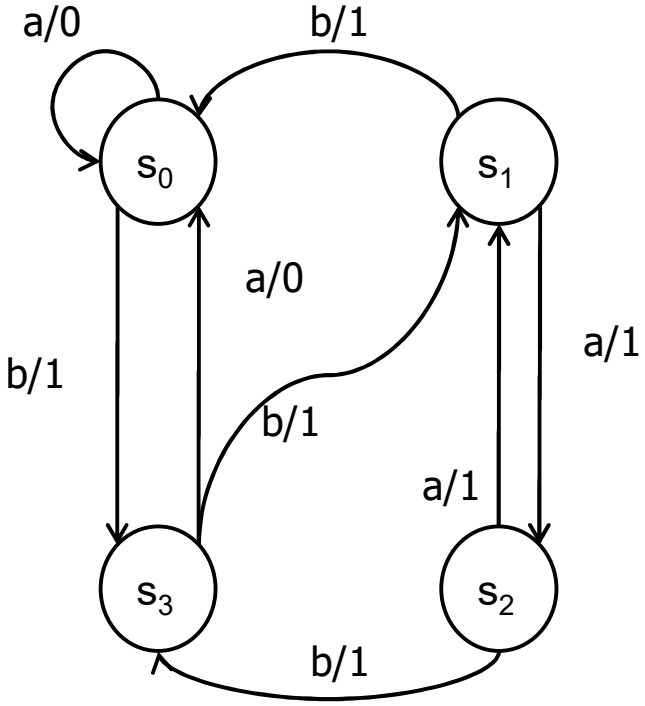
Summing up

- Why is it worth looking for a DS or UIOs rather than a W-set ?
- Hint: why did we start looking for such sequences in testing ?

Testing: the W-method (Vasilievski 1973)

- Given: a minimal machine M, with:
  - W : a characterizing set
  - P : set of sequences covering all transitions (spanning tree) and containing the empty sequence ε.
- Test suite T = P.W
- Implementation assumed to have a reliable reset "r" that brings it back to its initial state. r is prefixed to each test seq.

Example: W-method



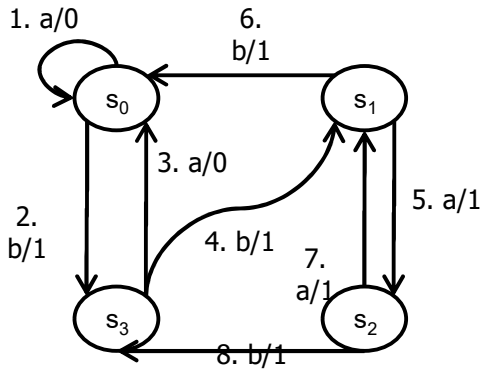
W = {a , bba}
P = {ε, a, b, ba, bb, bba, bbb, bbaa, bbab}

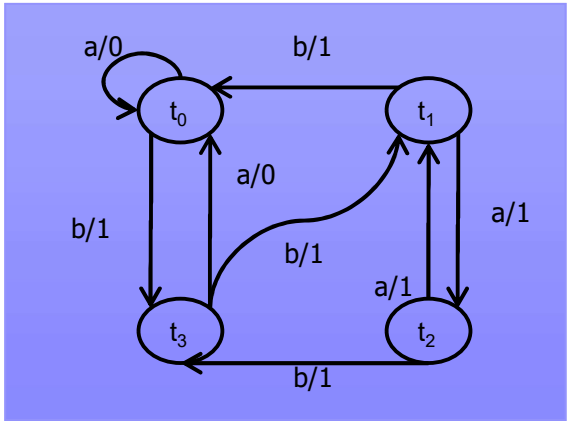
T = P.W
=
{a,bba,aa,abba,ba,bbba,baa,babba,bba,bbbba,
bbaa,bbabba,bbba,bbbbba,bbaaa,bbaabba,
bbaba,bbabbba}

Note: 7=longest non-cycling path
|T|= 9x2=18 tests (instead of 2^8-1=255)

Can be reduced by removing proper prefixes:
e.g: bba will be tested by bbabbba
T' ={aa,abba,baa,babba,bbbba,bbabba,bbba,
bbbbba,bbaaa,bbaabba,bbaba,bbabbba}
So only 12 tests needed

Understanding the W-method





Test suite:

ra/0,rb/1b/1a/1, Checks that $t_0 \approx s_0$

r a/0 a/0, r a/0 b/1b/1a/1 Checks output + tail state of 1 $\approx s_0$

r b/1 a, r b/1 bba, Checks output + tail state of 2 $\approx s_3$

rb a a, rb a bba, rb=transfer sequence to s_3
then check transition 3 (output + tail $\approx s_0$)

rb b a, rb b bba, transfer to s_3 then check tr 4, tail $\approx s_1$

rbb a a,rbb a bba, transfer to s_1 then check tr 5, tail $\approx s_2$

rbb b a, rbb b bba, transfer to s_1 then check tr 6

rbba a a, rbba a bba, transfer to s_2 then check tr 7

rbba b a, rbba b bba} transfer to s_2 then check tr 8

Completeness theorem (Vasilievskii 1973)

- Hypotheses on specification M
  - M is minimal
  - W : a characterizing set
  - P : spanning sequences (covering trans.)
- Hypotheses on SUT (implementation)
  - SUT has a reliable reset r
  - Nb_states(SUT) ≤ Nb_states(M) = n
- Theorem: SUT $\approx_{P.W}$ M $\Leftrightarrow$ SUT ≈ M

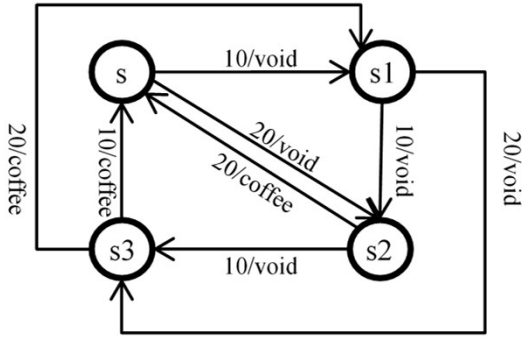
Travail pratique (mais pas que) à faire

- Chamilo, répertoire *TP-FSM*
- TP illustrant un article disponible dans répertoire *Lectures*
- Code incomplet fourni (Java)

- Objectif du TP :
  - Comprendre, par la pratique, les principales notions théoriques autour du test de conformité dans le cas des FSM (en complétant le code) – certaines d'entre elles ne sont pas vues en cours.
  - Me faire part de votre avis et des éventuels bugs (il y en a) ☺
- Retour attendu
  - Un court rapport précisant vos réponses aux questions (coller le code que vous avez écrit) et des éventuelles suggestions d'amélioration

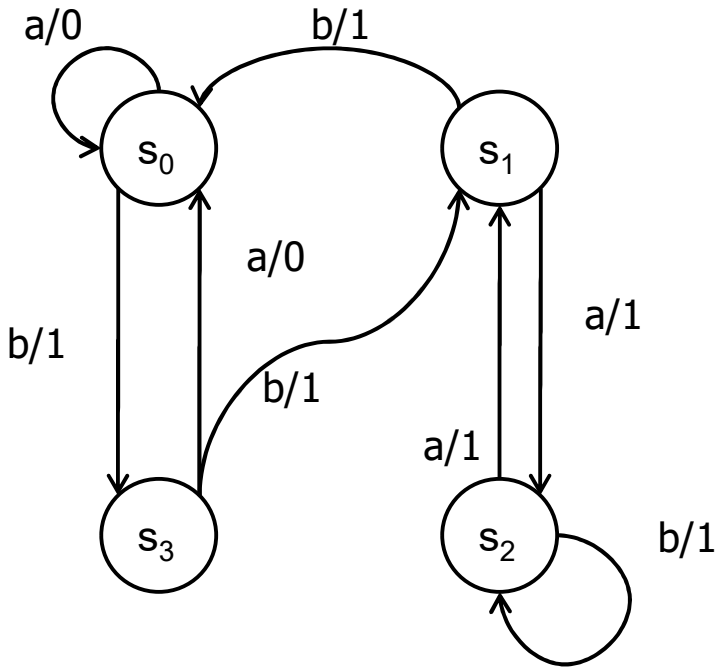
Exercice

- Considérons la spécification ci-dessous (*état initial s, existence de reset*).
- Construire un ensemble de séquences de test assurant que toute transition dans l'implémentation ne produit pas d'erreur de sortie ni erreur de transfert.



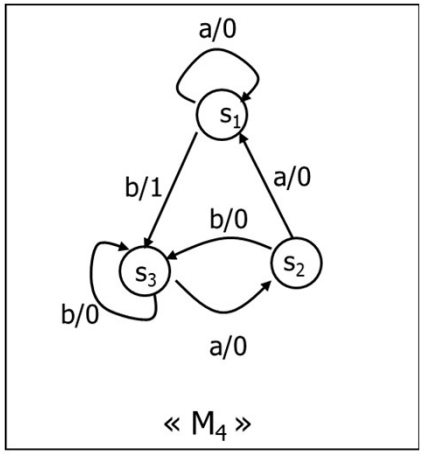
Exercice

- Considérons la spécification ci-dessous (*état initial s_0, existence de reset*).
- Construire un ensemble de séquences de test assurant que toute transition dans l'implémentation ne produit pas d'erreur de sortie ni erreur de transfert.



Exercice

- Considérons la spécification ci-dessous (*état initial s_1, existence de reset*).
- Construire un ensemble de séquences de test assurant que toute transition dans l'implémentation ne produit pas d'erreur de sortie ni erreur de transfert.



Extensions on FSM testing

- Algorithms to compute DS, UIO, W
- Identifying the starting state (if no reset)
  - Homing and synchronizing sequences
  - Test methods when no reset
- Testing Non-Deterministic Systems
  - preset tests with INCONCLUSIVE verdicts
  - adaptive tests

Other aspects of MBT

- So far: control part of model
  - Inputs a, b: can be parametrized: e.g. TCP_SYN(PortS,PortD,Seq,Ack,WSS…)
  - Transitions: decided by values, and update internal variables
- Addressing the data part: choice of parameter values
- Constraint solving (SAT, SMT…): a major field of Test Automation nowadays

Examples of MBT tools

- State models + Constraint solving
  - CertifyIT, YEST - Smartesting (France)
  - Qtronic - Conformiq (Finland)
  - Rational – IBM (techno from Sweden & Israel)
  - Spec Explorer – MicroSoft (USA)
- Statistical
  - Matelo - All4TEc (France)
  - JUMBL – U. Tenessee (USA)
- Combinatorial (data part): AETG

Point d'étape : comment on teste avec ce qu'on a appris?

1. On part d'une spécification sous la forme d'une FSM avec reset
2. On définit convenablement les vocabulaires d'entrée et de sortie
3. On vérifie les hypothèses (déterminisme, minimalité)
4. On détermine une DS ou bien un UIO par état ou bien un Wset
5. Pour chaque transition, on exécute une (ou plusieurs, si Wset) séquence de tests qui:
   i. Après reset, atteint l'état de départ de la transition
   ii. Exécute la transition et vérifie que la sortie et correcte
   iii. Poursuit l'exécution avec DS/UIO/Wset et vérifie que les sorties obtenues identifient bien l'état d'arrivée
6. L'oracle de tests est compris dans la méthode